

Tests in features interactions faults in IoT systems

João Guilherme Mendanha Alves

Who i'm ?

- Graduated with a technical degree in Industrial Automation from IFMG.
- Graduated in Information Systems from UFOP.
- Software Engineer at Levty.
- joao.mendanha.a@gmail.com
- <https://www.linkedin.com/in/joao-guilherme-mendanha-alves-2a1a79221/?skipRedirect=true>

Academic works

- Jogos com tecnologia vestível como estímulo à saúde dos pés – avaliação de usabilidade.
- Journal of Health Informatics.
- Evaluate the usability of digital games controlled by sensors attached to a sandal (wearable device)
- Analyze relevant aspects for their use in rehabilitation and maintenance of physical activity levels.
- Classificação de Notícias em Português Utilizando Modelos Baseados em Transferência de Aprendizagem e Transformers.
- Simpósio Brasileiro de Tecnologia da Informação e da Linguagem Humana.
- Classify the veracity of tweets in Brazilian Portuguese (PT-BR)
- Apply transfer learning and transformer-based models

Areas of interest

- Software Engineer.
- IoT Systems.
- Software testing.
- Bugs detection.

Definition: IoT's Systems

- A network of physical devices ("things") that communicate and exchange data over the Internet, distributed across multiple layers.

Examples of IoT



Smart Refrigerators



Smart Locks



Smart Lighting



Smart Plugs



Smart Mirrors



Smart Speakers



Smart Beds



Smart TVs



Smart Scales

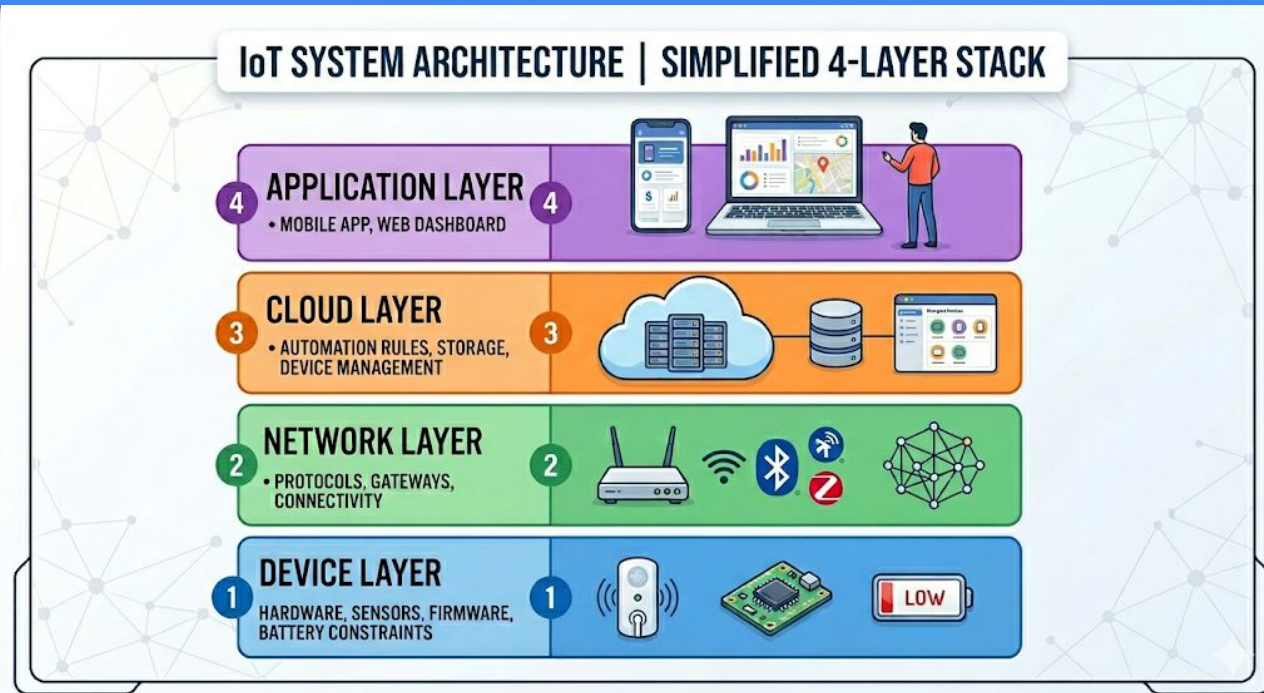


Smart Glasses

Definition: Multiple layers of configuration in IoT

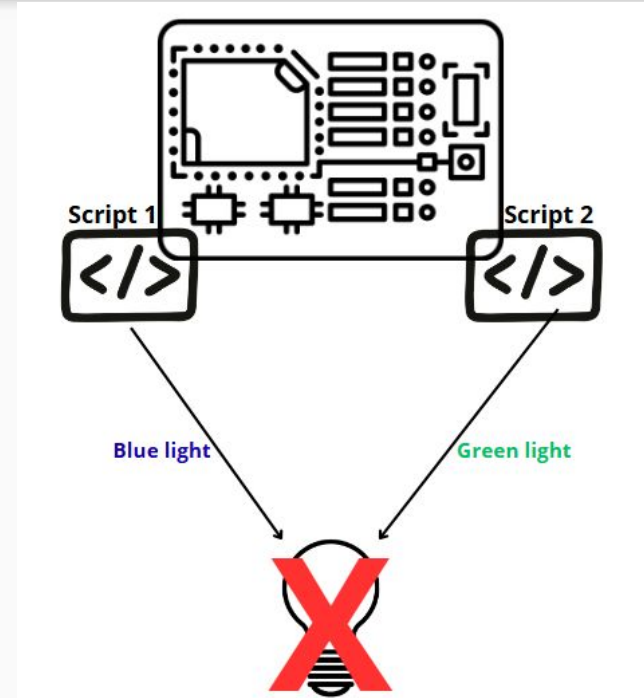
- IoT Layers are composed by:
 - Device layer: Hardware, sensors, firmware, battery constraints.
 - Network layer: Protocols, gateways, connectivity.
 - Cloud layer: Automation rules, storage, device management.
 - Application layer: Mobile app, web dashboard.
- Each layer adds configurable features that interact across boundaries.

Definition: Multiple layers of configuration in IoT



Definition: Features and Interaction Fault

- An atomic unit of functionality that can be enabled or disabled in a system.
- Unexpected behavior or failure that only manifests when two or more features are active simultaneously.



Motivations: Devices and new configurations

- 75 billion connected devices by 2025.
- Heterogeneity of devices, protocols, and software creates extreme variability.
- Failures that only appear in specific combinations of hardware/firmware/settings.
- Demonstrate why testing an IoT ecosystem is complex.

Motivations: Multiple layers of configuration in IoT

- A fault that only occurs when Feature A and Feature B are active simultaneously.
- Even in pure software, systems like checkstyle have 2^{135} valid configurations.
- In IoT the number of combinations is multiplied by device models, firmware versions and network conditions.
- Consequence is the combinatorial explosion is cited as a critical barrier by 80% of IoT testers.

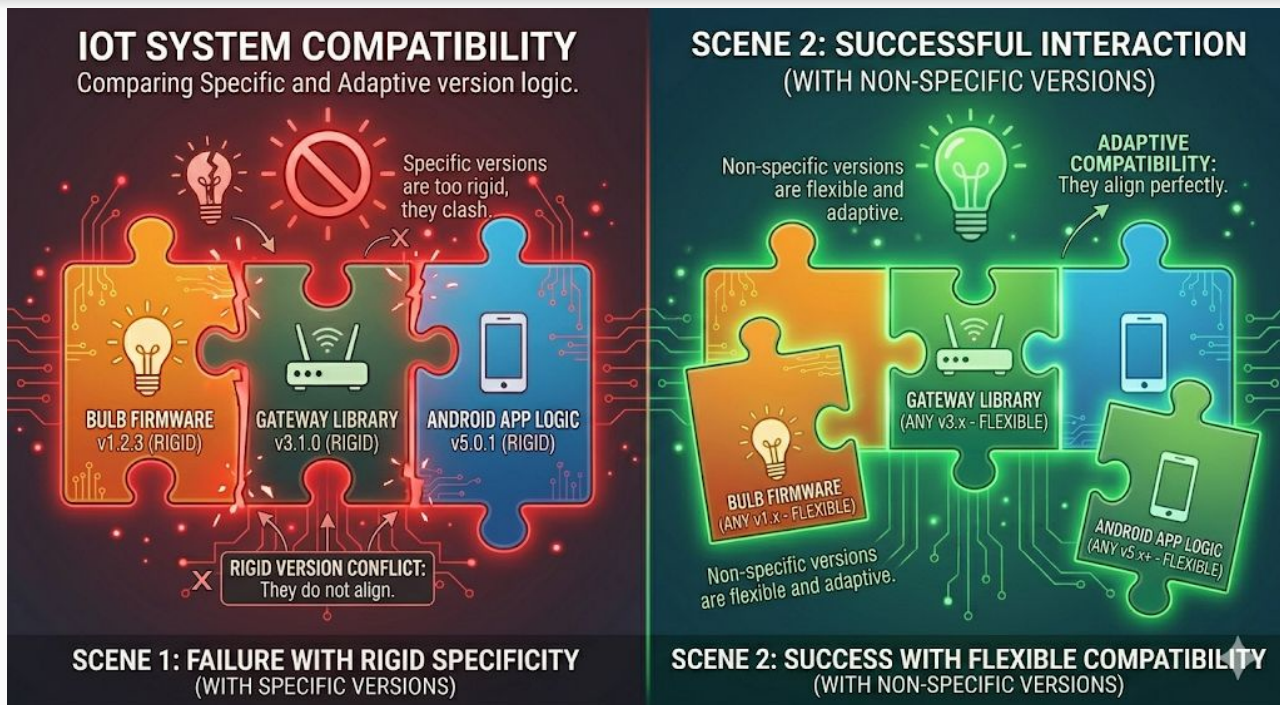
Motivations: Multiple layers of configuration in IoT

- $D \times F \times R \times (2^n)$.
- Assume an IoT system with:
 - $D = 5$ device models.
 - $F = 3$ firmware versions.
 - $R = 4$ network conditions.
 - $n = 20$ configurable software features.
- $5 \times 3 \times 4 \times 2^{20} = 62.914.560$.
- Disregarding device models: 12.582.912.

Motivations: Feature Interaction Faults

- IoT evidence:
 - Real Case (PYTRADFRI/135): An IKEA Trådfri bulb is misidentified as a sensor only when paired with a specific gateway firmware.
 - Root Cause: Interaction between Bulb Firmware + Gateway Library + Android App logic.

Motivations: Feature Interaction Faults



Related works

- Evaluating T-wise Testing Strategies in a Community-wide Dataset of Configurable Software Systems.
- IoT Bugs and Development Challenges.
- IoT systems testing: Taxonomy, empirical findings, and recommendations.

Evaluating T-wise Testing Strategies in a Community-wide Dataset of Configurable Software Systems.

- Focus: Comparison of 16 sampling strategies (CASA, Chvatal, ICPL, etc.) across 30 configurable software systems.
- Method: Empirical evaluation of speed, coverage, and fault detection effectiveness.
- Key Finding: Even the best 4-wise strategy found only ~30% of known faults; faults are concentrated in 0.8% of classes.

IoT Bugs and Development Challenges.

- Focus: First systematic study of real-world IoT failures based on 5,565 bug reports from 91 GitHub repositories.
- Method: Manual categorization of 323 bugs + interviews with 9 IoT experts + survey of 194 developers.
- Key Finding: Connectivity issues are the most frequent and severe; developers rely heavily on manual testing with physical devices.

IoT systems testing: Taxonomy, empirical findings, and recommendations.

- Focus: A structured taxonomy covering 7 dimensions of IoT testing (Objectives, Tools, Testers, Testing stages, Testing environment, Testing approaches, metrics).
- Method: Systematic review of 83 studies + two-round validation survey with 220 industry practitioners.
- Key Finding: The taxonomy helps testers identify significantly more test scenarios and non-functional aspects, especially End-to-End interactions.

Lessons learned: Where faults hide in IoT ?

- Based on IoT Bug Taxonomy (Makhshari & Mesbah):
 - Connectivity: Device fails to reconnect after network drop.
 - Device management: Failure in pairing (Binding) or Over-the-Air (OTA) firmware updates.
 - Messaging: Malformed payloads or incorrect command ordering.
- An aggravating factors is the timing issues and environmental dependencies.

Lessons learned: How to do to detect IoT bugs and features interaction faults ?

- Ferreira proposes: Prioritize Classes and Features with high Complexity (WMC) and Scattering.
- Makhshari suggests: Focus testing on connectivity and OTA Updates (root causes of Timing/Dependency faults).
- Minani recommends: End-to-End (E2E) Testing that spans all architecture layers.
- Test fewer configurations, but target more dangerous configurations.

Testing tools

Category	Key Tools	Primary Use
Device & Fleet Simulation	IoTIFY, QEMU, Renode, SimulIDE, Digi IoT Application Framework	Simulate thousands of virtual devices; test firmware before hardware deployment
Service & API Virtualization	Parasoft Virtualize, Postman, HiveMQ	Mock cloud/edge dependencies; validate MQTT and HTTP APIs
Load & Performance Testing	Apache JMeter, Gatling, LoadFocus IoT Suite, Katalon Studio	Simulate millions of concurrent device connections; stress-test backend systems

Testing tools

Category	Key Tools	Primary Use
Protocol & Connectivity Diagnostics	Wireshark (with MQTT/CoAP filters), MQTTBox, Mosquitto_pub/sub	Debug packet loss, handshake failures, and protocol mismatches
Firmware & Security Testing	Finite State, Binwalk, ProtoCrawler, IoT HackBot, TELEMETRY	Static/dynamic firmware analysis; protocol fuzzing; credential scanning

Testing tools

Category	Key Tools	Primary Use
Continuous Monitoring & End-to-End Frameworks	Thoth, TestGrid (CoTester), AWS IoT Device Tester, Eclipse IoT Testware	End-to-end validation across device-gateway-cloud; AI-assisted test generation
Hardware-in-the-Loop (HIL) & RF Testing	NOFFZ ITN 1200, rf-testbench (LoRa/BLE/Wi-Fi)	Validate real hardware behavior; measure RSSI, packet loss, and frequency drift

Conclusion

- High variability of hardware/software makes IoT the most challenging domain for configuration testing.
- Interaction failures are the norm, not the exception.
- A strategy is needed that tests hardware and software and their interactions.

Project methodology

- Find or build a database with interaction faults and bugs.
- Write test cases.
 - Using a LLM's models.
- Apply strategies to optimize the tests cases combinations.
 - Based in literature recommendations
- Use a testing tools to automatize the tests.
- Write guidelines to testing IoT Systems.

Bibliographic references

- Makhshari, Amir, and Ali Mesbah. "IoT bugs and development challenges." 2021 IEEE/ACM 43rd International Conference on Software Engineering (ICSE). IEEE, 2021.
- Minani, Jean Baptiste, et al. "IoT systems testing: Taxonomy, empirical findings, and recommendations." Journal of Systems and Software 226 (2025): 112408.
- Ferreira, Fischer, et al. "Evaluating T-wise testing strategies in a community-wide dataset of configurable software systems." Journal of Systems and Software 179 (2021): 110990.

Questions ?